

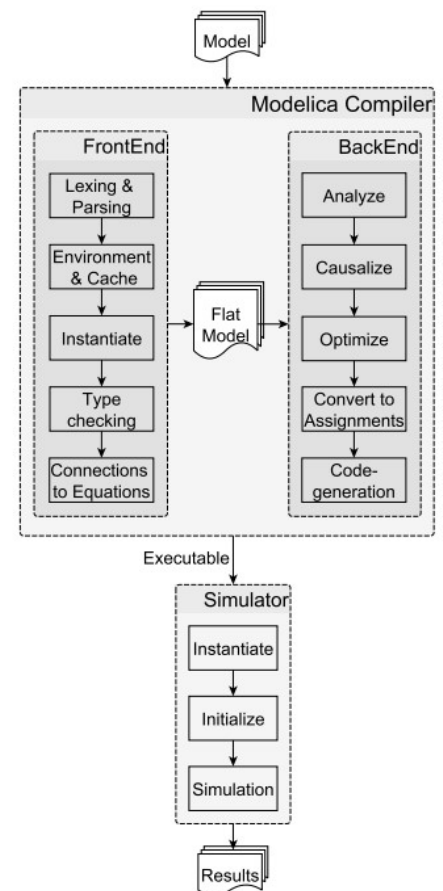
2 - Analysing Modelica

Donnerstag, 24. Juli 2025 15:34

Datum	24.07.2025
Kategorie	Tool Analysis
Quelle	Principles of Object-Oriented Modelling and Simulation with Modelica, OpenModelica Compiler — OpenModelica User's Guide v1.26.0-dev-207-g710d7e10735 documentation
Arbeitspaket	AP4: Analyse bestehender Tools

Implementation and Execution of a Modelica Model

1. Modelica source code is parsed and converted into an internal representation
2. Representation is analyzed:
 - Type checking
 - Classes are inherited and expanded
 - Modifications and instantiations are performed
 - Connect equations are converted to standard equations
 - **Result is a flat set of equations, constants, variables, and function definitions**
 - No trace of object-oriented structure remains
3. All of the equations are topologically sorted according to the data-flow dependencies between the equations
 - In case of **DAEs: sorting, manipulation of equations to convert the coefficient matrix into block lower triangular form**
 - **Optimizer module: algebraic simplification algorithms, symbolic index reduction methods**
 - Elimination of most of the equations
 - **Keeping minimal set of equations that will be solved numerically**
4. **Independent equations in explicit form are converted to assignment statements**
 - Possible since the equations have been sorted and an execution order has been established for evaluation of the equations in conjunction with the iteration steps of the numeric solver
 - **Strongly connected set of equation: set is transformed by a symbolic solver** -> performs a number of algebraic transformations to simplify the dependencies between the variables (can solve a differential equation if it has a symbolic solution)
5. **C code is generated and linked with a numeric equation solver** that solves the remaining, drastically reduced, equation system
6. **Approximations to initial values are taken from the model definition** or are interactively specified by the user
 - Numeric solver for DAEs computes the values of the variables during the specified simulation interval
 - Result of the dynamic system simulation is a set of functions of time
 - Functions can be displayed as graphs and/or saved



Principles of Object-Oriented Modelling and Simulation with Modelica

Mittwoch, 25. Juni 2025 23:07

Datum	25.06.2025 bis 26.06.2025
Kategorie	Tool Analysis / Framework Inspiration
Quelle	Principles of Object-Oriented Modelling and Simulation with Modelica
Arbeitspaket	AP4: Analyse bestehender Tools

Summary

- **Core Language Features**
 - **Equation-based modeling:** Focus on acausal relationships (no fixed data flow), enhancing flexibility and reusability
 - **Multi-domain modeling:** seamless integration of physical domains (electrical, mechanical, hydraulic, thermal)
 - **Object-oriented design:** enables hierarchical structuring, inheritance, encapsulation, and reuse
 - Component-based modeling: well defined interfaces (connectors) and strong typing
- **Modeling constructs**
 - **Classes and Instances:** all models are classes; instantiation is implicit. Equations, not methods define the behavior.
 - **Specialized classes:** model, block, connector, partial, function, etc.; each with specific semantic
 - **Partial classes & inheritance:** facilitates reuse and abstraction in component libraries
 - **Generic classes & replaceable components:** enables parameterized, flexible model architecture
- **Modeling semantics:**
 - **Acausal Physical Modeling:** direction of computation is determined at compile time
 - **Equations:** multiple types (declaration, modification, initial, normal), always represent equality not assignment
 - **Hybrid Modeling:** Support of discrete events (when equations, state machines, synchronous clock)
- **Component and Connection Mechanism:**
 - **Connectors:** Carry flow and potential variables (current and voltage) and define physical interface
 - **Connections:** Generate equations (Kirchhoff-style) automatically
 - **Expandable Connectors:** act like information buses; allow scalable flexible component interaction
 - **Stream connectors:** For bi-directional flow with properties (e.g. enthalpy, composition)
- **Execution Pipeline:**
 - a. Parsing and flattening into DAE system
 - b. Topological sorting and symbolic transformation (index reduction)
 - c. Generation of optimized C code and linking with a numerical solver

Relevant Concepts:

Equation-based acausal modeling	• crucial for enabling modular subsystem exchange without imposing a fixed data flow • aligns with need to support heterogeneous simulation paradigms
Component Interface & Connection Concept	• Orientation on Modelica's connector concept (flow/potential) for defining clear I/O interfaces • Connection rules (sum-to-zero, equality) model physical conservation laws (coupling mechanisms)
Partial & Generic Class Mechanisms	• Template-based subsystem definitions in the frameworks • Motor, sensor, controller templates
Hybrid & Multi-Domain	• critical for modeling prosthetic systems involving mixed

Modelling	continuous/discrete dynamics • Sensor logic and mechanical actuation
Simulation Architecture & Compilation Pipeline	• Modelica's approach to flattening, index reduction, and code generation illustrates how to handle system equations systematically • Modular code generation, symbolic processing, solver integration strategies for framework
Reusable Component Libraries	• Structured component libraries can guide how to organize reusable, parameterized, subsystem models
Diagrammatic Representation & Connectors	• Graphical modeling with connection diagrams • support for hierarchy visualization • Visualization of system structure and simulation flow

Notes

Chapter 2 - A quick Tour of Modelica

- Modelica is an object-oriented, equation-based programming language oriented toward computational applications with high complexity requiring high performance

Four most important features:

- **Equations-based modeling**
 - Instead of assignment statements
 - Permits **acausal modeling** that gives better reuse of classes
 - Equations do not specify a certain data-flow direction
 - Modelica class can adapt more than one data flow context
- **Multidomain modeling capability**
 - Model components corresponding to physical **objects from several domains**
 - Electrical, mechanical, thermodynamic, hydraulic, biological, control applications
 - Model components can be described and connected
- **Object-oriented language with general class concept**
 - Unifies classes, generics (similar to C++ templates), and general subtyping into a single language construct
 - Facilitates reuse of components and evolution of models
- **Strong software component model**
 - Constructs creating and connecting components
 - Language is ideally suited as an architectural description language for complex physical systems and, to some extent, to software systems

2.1 Getting Started with Modelica

- Modelica **programs are built from classes (also called models)**
- Class as collection of blueprints and instructions
- Class contains elements that can be variable declarations and equation sections containing equation
- Variables contain data belonging to instances of the class (data storage of the instance)
- Equations of a class specify the behavior of instances of the class

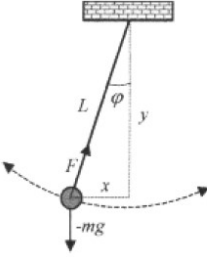
First Example:

<pre>class HelloWorld Real x(start = 1); parameter Real a = 1; equation der(x) = -a*x; end HelloWorld;</pre>	• Variable x is a dynamic variable (here also a state variable) • der(x) is the time derivative of x • Hello world program is declared as class since all programs are declared as class • All variables have a start attribute with a default value that is set to 0
--	---

- Different start value is accomplished by providing a so-called modifier within parentheses after the variable name
 - Modification equation setting the start attribute to 1
 - Constant is prefixed with keyword parameter -> is constant during simulation
 - Parameter "a" can be changed through command in simulation environment

- Invocation of the simulation compiles the simulation to C code which is compiled to machine code
 - Simulation is executed together with ODE or DAE solve
 - `simulate(HelloWorld,stopTime=2)`
 - `plot(x)`
- Program contains a declaration of a class with 2 variables and a single equation
 - Attribute variable `x` -> initialized to 1
 - Attribute variable `a` -> constant initialized to 1 at the beginning of the simulation
 - Constant is indicated by keyword *parameter* but can be changed between simulations
- Each variable has a name and a type when it is declared
- In modelica, the equal sign = always means equality and not an assignment as in other languages
- Time derivative is indicated by the pseudofunction `der(...)`

Second Example:

$m\dot{v}_x = -\frac{x}{L}F$ $m\dot{v}_y = -\frac{y}{L}F - mg$ $\dot{x} = v_x$ $\dot{y} = v_y$ $x^2 + y^2 = L^2$		<pre> class Pendulum "Planar Pendulum" constant Real PI=3.141592653589793; parameter Real m=1, g=9.81, L=0.5; Real F; output Real x(start=0.5), y(start=0); output Real vx, vy; equation m*der(vx)=- (x/L) * F; m*der(vy)=- (y/L) * F - m*g; der(x)=vx; der(y)=vy; x^2+y^2=L^2; end Pendulum; </pre>
--	---	--

- Newton's Equation of motion for the pendulum mass under the influence of the gravity
 - First 4 equations are differential equations
- Geometric Constraint: $x^2+y^2=L^2$ -> algebraic equation
 - Only involving algebraic formulas of variables but no derivatives
- Velocities and positions in x and y directions
- Systems that contain both differential and algebraic equations are called differential algebraic equation systems (DAEs)
- Number of variables must be equal to the number of equations

2.1.1 Variables and Predefined Types

- **Parameter:** constant variable that can be modified by a user
- **Constant:** never changes and can be substituted by its value
- Predefined basic data types:

Boolean	either true or false
Integer	32-bit two's complement, usually corresponding to the C int data type
Real	64-bit floating-point usually corresponding to the C double data type
String	string of text characters
enumeration{...}	enumeration type of enumeration literals
Clock	clock type for clocked synchronous models
Complex	for complex number computations, a basic type predefined in a library

2.1.2 Comments

- "a comment":
 - optionally appearing after a variable declaration or at beginning of class declaration
 - Appear in menus or as help text for user
- "// Comment": same line
- /* and the text */: larger comment sections

2.1.3 Constants

- Named constants with keyword constant or parameter
- Parameters can usually be accessed and changed from modelics tool
 - Resimulation without recompilation
- Parameters can be declared without a declaration equation
 - Can be defined by reading from file
- Constant are fixed after declaration

2.1.4 Variability

- Expressions or variables with continuous-time variability can change at any point in time.
- Discrete-time variability: values can change only at so-called events
 - Unlocked discrete-time
 - Clocked discrete-time variability -> can only change at clock tick events
- Parameter variability: value can be changed at initialization before simulation but is fixed during
- Constant variability: value is always fixed

2.2 Object-Oriented Mathematical Modeling

- **Inspired by mathematics: common to state what holds** rather than giving detailed stepwise algorithm on how to achieve the desired goal
- Object-orientation is primarily used as a structuring concept
 - Emphasizing declarative structure and reuse of models
 - Ways of structuring: hierarchies, component connections, and inheritance
- Dynamic properties are expressed in a declarative way through equations
- An object is a collection of instance variables and equation that share a set of data
- Is not viewed as dynamic message passing
- Classes are blueprints for creating objects
- Variables and equations can be inherited between classes
- Specifying behavior is primarily done through equations instead via methods

2.3 Classes and Instances

- Class **members**
 - **Data variables**: results of computations caused by solving equations from class together with equations from other classes
 - **Equations** specify the behavior of a class
 - **Classes** can be members of other classes
- Member **access using dot(.) notation**
- Visibility of class members **by default public**
 - Protected: only code inside the class as well as code in classes that inherit this class are allowed to access

2.3.1 Creating Instances

- Objects are created implicitly just by declaring instances of classes
- Class at the top of the instantiation hierarchy in the modelica program is executed as kind of main class that is always implicitly instantiated
- No class variables in modelica

2.3.2 Initialization

- Start values are set by setting the start attribute of instance variables
- More general way of initialization: *initial equation*
 - Specification of a equation system to be solved in order to obtain the initial values of these variables

Initial equation	Solution at initialization
<pre> model Controller parameter Real a=1, b=2; Real y; equation der(y) = a*y + b*u; initial equation der(y)=0; end Controller; </pre>	<pre> der(y) = 0; y = -(b/a)*u; </pre>

2.3.3 Specialized Classes

- **Model** class
 - Is almost the same as a class, keywords are interchangeable
- **Block** class
 - Class with a fixed causality
 - For each connector variable of the class it is specified whether it has input or output causality
 - Connector variable must be declared with a causality prefix keyword of either input or output

- **Connector** class
 - Used to declare the structure of ports or interface points of a component
 - May not contain equations
 - Has only public sections
 - Has the additional property to allow connect(...) to instances of connector classes
- **Type** class
 - Can be an alias or an extension to a predefined type, enumeration, or array of a type
 - Example: *type vector3D = Real[3]*
- **Package** and **function** classes
 - Functions can be called with an argument list and instantiated at runtime
- **Operator** class
 - Like package class a container for other definitions
 - May only contain functions and is intended for userdefined overloaded operators
- **Idea of specialized classes:**
 - Advantages since they reuse most properties of the general class concept
 - Gives the user a chance to express more precisely what a class is intended for
 - Requires the modelica compiler to check that these usage constraints are fulfilled

2.3.4 Reuse of Classes by Modifications

- Connection of two filter classes by creating 2 appropriately modified instances of this class that are connected

<pre> model LowPassFilter parameter Real T=1 "Time constant of filter"; Real u, y(start=1); equation T*der(y) + y = u; end LowPassFilter; </pre>	<pre> model FiltersInSeries LowPassFilter F1(T=2), F2(T=3); equation F1.u = sin(time); F2.u = F1.y; end FiltersInSeries; </pre>
--	--

- Connecting by equation: $F2.u = F1.y$
- Time variable is a built-in, predefined variable *time*

2.3.5 Built-In Classes

Types: Real, Integer, Boolean, String, enumeration(...), Clock

- Can be inherited, modified, etc.
- Only the value attribute can be changed at runtime and is accessed through the variable name itself: x instead of x.value
- Example: *class Voltage = Real(unit="V", min=-220.0, max=220.0);*

2.4 Inheritance

- Original class: superclass, base class -> is extended to create a more specialized version
 - Subclass, derived class
- Behavior and properties are reused

<pre> record ColorData Real red; Real blue; Real green; end ColorData; </pre>	<pre> class Color extends ColorData; equation red + blue + green = 1; end Color; </pre>	<pre> class Point public Real x; Real y, z; end Point; </pre>	<pre> class ColoredPoint extends Point; extends Color; end ColoredPoint; </pre>
---	---	--	---

2.5 Generic Classes

- Similar to templates in C++
- **Class parameters as instance** parameters: instance declarations as values
- **Class parameters as type** parameters: have types as values
- **Formal parameters are prefixed by keyword *replaceable***
- Equivalent to virtual methods in C++

2.5.1 Class Parameters Being Instances

- Class parameters -> replaceable -> with default type
- Variable -> without replaceable
- **Redeclare:** precedes an actual argument to a class formal parameter to change its type
 - Keyword to avoid accidentally changing the type
- Type constraints are possible

2.5.2 Class Parameters Being Types

- Parameter can also be a type

- Useful for changing the type of many objects

2.6 Equations

- Equations do not describe a certain data flow direction or execution order
- In Modelica the following holds
 - **Assignment statements are usually represented as equations in modelica**
 - Attribute assignments are represented as equations
 - **Connections between objects generate equations**
- Classification of modelica equations:
 - **Normal** equations: in equation section including connect equation
 - **Declaration** equations: part of variable / constant declarations
 - Always holds
 - **Modification** equations: used to modify attributes
 - **Initial** equations: specified in initial equation sections or as start attribute equations
 - Used to solve the initialization problem at startup time

2.6.1 Repetitive Equation Structures

- Algorithmic for-loop to set up polynomial equations with array indexing for getting the coefficients
- Compact vector notation possible: $xpowers[2:n+1] = xpowers[1:n]*x;$

2.6.2 PDEs

- contain derivatives with respect to other variables than time
- PDE functionality is not yet part of the official Modelica language.

2.7 Acausal Physical Modeling

- Modeling based on equations instead of assignment statements
- Inputs/outputs are not specified by the equations
- Causality of equation-based models is unspecified
- Acausal: becomes fixed only when the corresponding equation system is solved
- Very well suited for representing the physical structure of modeled systems
- **Advantage: Solution direction of equations will adapt to the data flow context in which the solution is computed**
- Data-flow context is defined by stating which variables are needed as output and which are external inputs to the system

2.7.1 Physical Modeling vs. Block-Oriented Modeling

- Connections between objects: **connect equation construct -> special syntactic form of equation**
- **Block-oriented** model is **causal** since the signal **flow has been deduced and is clearly shown in the diagram**
 - Conversion to block model is non-trivial
 - Resistor representation are context dependent
 - Hard to maintain even for small changes

2.8 The Modelica Software Component Model

Items of a software component model:

1. **Components**
2. **A connection mechanism**
3. **A component framework**

- Components are connected via the connection mechanism and can be visualized in connection diagrams
- Component framework realizes components and connections
 - Ensures that communication works and constraints are maintained over the connections
- Systems of acausal components: data flow direction is automatically deduced by the compiler at composition time

2.8.1 Components

- Simple instances of modelica classes with well-defined interfaces (ports/connectors) for communication and coupling between components

- Is modeled independently of the environment in which it is used
- Only local variables and connectors can be used
- **Communication is only allowed via connectors**
- Access of component data via dot notation is also possible -> component consists of other components -> hierarchical modelling

2.8.2 Connection Diagrams

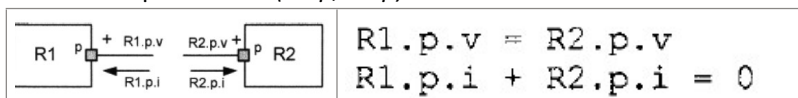
- Pictorial representation of components and connections to grasp complexity
- Lines represent physical connections
- Connectors are shown as small square dots on the rectangles
 - Variables at such interface points define the interaction between the components

2.8.3 Connectors and Connector Classes

- Connectors are instances of connector classes which define the variables that are part of the communication interface that is specified by a connector
- Pin connector class
 - Flow prefix: variable represents a flow quantity

2.8.4 Connections

- Connections require connectors of equivalent type
- **Types of coupling depending if potential or flow:**
 - **Equality coupling for potential variables: Kirchhoff's first law**
 - **Sum-to-zero coupling for flow variables: Kirchhoff's current law**
- Flow for current in pin connector -> all currents in connected pins are summed to zero
- Example: `connect(R1.p, R2.p)`



- Voltages of connected wires are the same
- Currents sum to zero at a node
- Implicit connections for modeling force fields with inner/outer construct

2.8.5 Implicit Connections with inner/outer

- Force field: connection might be needed between each pair of components
- 1xn connection: implicit connections between an object and n of its components through inner and outer declaration prefixes
- Shared attribute as common kind of implicit interaction
 - Shared environment temperature

```

model Environment
  inner Real T0; //Definition of actual environment temperature T0
  Component comp1, comp2; //Lookup match comp1.T0 = comp2.T0 = T0
  parameter Real k=1;
equation
  T0 = sin(k*time);
end Environment;

model Component
  outer Real T0; // Reference to temperature T0 defined in the environments
  Real T;
equation
  T = T0;
end Component;

```

2.8.6 Expandable Connector for Information Buses

- Transporting information between various system components
- Expandable connector: acts like an **information bus**
- Intended for connection to many kind of components
- **Automatically expands the expandable connector type to accommodate all the components connected to it with their different interfaces**
- Fields in expandable connector are seen as connector instances
- Connection of 2: each is augmented with the variables that are only declared in the other expandable connector
 - Repeated until all expandable connector instances have matching variables
 - If a variable appears as an input in one connector it should appear as non-input in at least one expandable connector in the connected set

2.8.7 Stream Connectors

- Stream variable
- For bi-directional flows of matter with associated quantities
- Convective transport of specific quantities such as specific enthalpy or chemical composition

```
connector FluidPort
...
flow Real m_flow "Flow of matter; m_flow>0 if flow into component";
stream Real h_outflow "Specific variable in component if m_flow < 0"
end FluidPort

model FluidSystem
...
FluidComponent m1, m2, ..., mN;
FluidPort c1, c2, ..., cN;
equation
connect(m1.c, m2.c);
...
connect(m1.c, cN);
...
end FluidSystem;
```

2.9 Partial Classes

- Partial class does not contain enough equations to completely specify its physical behavior
- Is prefixed with the keyword **partial**
- Example: partial class TwoPin that captures the property of many electrical components that have two pins
- C++: similar to abstract classes since it cannot be used as a class to instantiate a data object

```
partial class TwoPin "Superclass of elements with two electrical pins"
Pin p, n;
Voltage v;
Current i;
equation
v = p.v - n.v;
0 = p.i + n.i;
i = p.i;
end TwoPin;
```

2.9.1 Reuse of Partial Classes

Resistor	Capacitor
<pre>class Resistor "Ideal electrical resistor" extends TwoPin; parameter Real R(unit="Ohm") "Resistance"; equation R*i = v; end Resistor;</pre>	<pre>class Capacitor "Ideal electrical capacitor" extends TwoPin; parameter Real C(unit="F") "Capacitance"; equation C*der(v) = i; end Capacitor;</pre>

- Current i and voltage v evolve at functions of time
- Solver of differential equations computes the values of $v(t)$ and $i(t)$ s.t. the constitutive equation are fulfilled for the capacitor

2.10 Component Library Design and Use

- Component libraries of reusable components are the key of effective modeling of complex systems

2.13 Arrays

- Collection of variables of the same type
- Integer indexes from 1 to n
- Array variable can be declared by appending dimensions `Real[3]`
- Range $i1:i2$ -> all indexed elements starting with $i1$ up to and including $i2$
- Creating $m \times n$ matrix:

```
[expr11, expr12, ... expr1n;
expr21, expr22, ... expr2n;
...
exprm1, exprm2, ... exprmn]
```

2.14 Algorithmic Constructs

- Procedural description of how to carry out specific computations

- Consisting of a number of statements that have to be carried out in a specified order

2.14.1 Algorithm Sections and Assignment Statements

- Algorithmic statements can only occur in algorithmic sections
- Keyword *algorithm*
- Can appear among equation sections
- Terminated by keywords: equation, public, protected, algorithm, end, annotation
- Algorithm uses variables from outside the section as input variables to the algorithm
- Variables assigned values by the the algorithm define outputs of the algorithm

2.14.2 Statements

<pre> sum := 0; n := size(a,1); while n>0 loop if a[n]>0 then sum := sum + a[n]; elseif a[n] > -1 then sum := sum - a[n] -1; else sum := sum - a[n]; end if; n := n-1; end while; </pre>	<pre> algorithm y := 0; xpower := 1; for i in 1:n+1 loop y := y + a[i]*xpower; xpower := xpower*x; end for; ... </pre>
---	--

2.14.3 Functions

- Modelica standard library Modelica.Math: abs, sqrt, mod, sin, cos, exp, ...
- Body of a function is an algorithm section that contains procedural algorithmic code to be executed when the function is called
 - Input: formal parameters
 - Outputs: result of the function

```

function polynomialEvaluator
  input Real a[:]; // Array, size defined at function call time
  input Real x := 1.0; // Default value 1.0 for x
  output Real y;
protected
  Real xpower;
algorithm
  y := 0;
  xpower := 1;
  for i in 1:size(a,1) loop
    y := y + a[i]*xpower;
    xpower := xpower*x;
  end for;
end polynomialEvaluator;

```

- Functional formal parameters are read-only -> may not be assigned values within the code of the function
- Function call
 - using positional argument association
 - Using named argument association
- Formal parameters with default values need not to be specified as actual arguments

```
(a, b, c) := f(1.0, 2.0);
```

In equations a similar syntax is used:

```
(a, b, c) = f(1.0, 2.0);
```

2.14.4 Operator Overloading and Complex Numbers

- Operators: + and * for complex numbers
- Solve() for different matrix representations
 - Sparse, standard dense, symmetric matrices
- Constructor *fromReal()* function converts real number to complex number

<pre> operator record Complex "Record defining a Complex number" Real re "Real part of complex number"; Real im "Imaginary part of complex number"; encapsulated operator 'constructor' import Complex; function fromReal input Real re; output Complex result = Complex(re=re, im=0.0); annotation(Inline=true); end fromReal; end 'constructor'; </pre>	<pre> encapsulated operator function '+' import Complex; input Complex c1; input Complex c2; output Complex result "Same as: c1 + c2"; annotation(Inline=true); algorithm result := Complex(c1.re + c2.re, c1.im + c2.im); end '+'; end Complex; </pre>
---	---

2.14.5 External Functions

- Possible to call functions outside of the modelica language (implemented in C(default) or Fortran)

2.14.6 Algorithms Viewed as Functions

- Algorithm within equation section can be transformed into an equation and a function without changing meaning
- Function has the inputs to the algorithm section as its formal parameters and the outputs as its result parameters

<pre> algorithm y := x; z := 2*y; y := z+y; ... </pre>	<pre> (y,z) = f(x); ... function f input Real x; output Real y,z; algorithm y := x; z := 2*y; y:= z+y; end f </pre>
---	--

2.15 Discrete-Event and Hybrid Modelling

- Macroscopical physical systems (mechanical, current and voltage) are said to have continuous dynamics
- Sometime beneficial to make approximation that a system displays discrete behavior
 - Changes of variables may occur instantaneously at specific point in time
 - Simplifies the mathematical model
 - Speeding up the simulation
- Possible to have variables of discrete time-variability
 - Clocked discrete-time variables: undefined between clock-ticks
 - Unclocked discrete-time variables: keep value constant in between ticks
- Example: *discrete Real*
- Can only be applied to certain subsystems -> model will consist of continuous and discrete models -> hybrid system and hybrid modelling
- Constructs for expressing hybrid models:
 - Conditional expressions or equations to describe discontinuous and conditional models
 - Equations are only valid at discontinuities
- When-equations -> conditional equation

when {condition1, condition2, ...} then

<equations>

end when;

- Bouncing ball:

```

model BouncingBall "Simple model of a bouncing ball"
  constant Real g = 9.81 "Gravity constant";
  parameter Real c = 0.9 "Coefficient of restitution";
  parameter Real radius=0.1 "Radius of the ball";
  Real height(start = 1) "Height of the ball center";
  Real velocity(start = 0) "Velocity of the ball";
  equation
    der(height) = velocity;
    der(velocity) = -g;
    when height <= radius then
      reinit(velocity, -c*pre(velocity));
    end when;
  end BouncingBall;

```

- Reinitialization of the ball velocity at bounce event

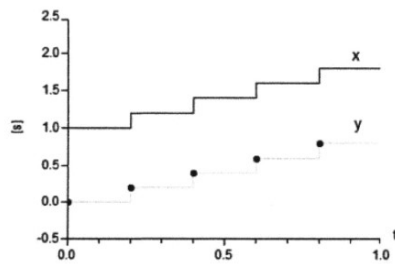
2.15.1 Synchronous Clocks and State Machines

- Built-in synchronous clocks with constructs for clock based modeling
- Synchronous clock-based state machines
- Synchronous clock-based modeling gives additional advantages for real-time and embedded system modeling in terms of higher performance and avoidance and/or detection of certain kinds of errors

```

model BasicClockedSynchronous
  Clock clk "A clock variable";
  discrete Real x "Clocked discrete-time variable";
  Real y "Unclocked discrete-time variable";
equation
  clk = Clock(0.2); // Clock with an interval of 0.2 seconds
  x = sample(time,clk); // Sample the continuous time at clock ticks
  y = hold(x)+1; // Hold value is defined also between clock ticks
end BasicClockedSynchronous;

```



2.16 Packages

- A **package is simply a container or a namespace for names** of classes, functions, constants, and other allowed definitions
- Package name is prefixed to all definitions in the package using standard dot notation
- Definitions can be imported into the name space of a package
- **Similar to C++ namespace**
- Well-designed **package should state all its dependencies explicitly through import statements** that are easy to find
- Encapsulated in front of package prevents nested lookup outside the package boundary

Bad	Better
<pre> package MyPack import MyTypes.*; import Modelica.SIunits.*; class Foo; Voltage v; Modelica.SIunits.Ampere i; end Foo; end MyPack; </pre>	<pre> encapsulated package MyPack import SI = Modelica.SIunits; import Modelica; class Foo; SI.Voltage v; Modelica.SIunits.Ampere i; end Foo; ... end MyPack; </pre>

2.17 Annotations

- Extra information associated with a Modelica Model
- Used by modelica environments for supporting documentation or graphical model editing
- Most annotations do not influence the execution of a simulation

2.18 Naming Conventions

- Type and class names (but not for functions) always start with an uppercase letter
- Variable names start with a lowercase letter, exception T for temperature
- Names consisting of several words have each word capitalized
- Underscore character is only used to characterize lower or upper indices: body_low_up
- Preferred names for connector instances in (partial) models are
 - p and n for positive and negative connectors in electrical components
 - a and b for other kinds of otherwise-identical connectors occurring in two-sided components

2.19 Standard Library

Modelica.Constants	Common constants from mathematics, physics, etc.
Modelica.Icons	Graphical layout of icon definitions used in several packages.
Modelica.Math	Definitions of common mathematical functions.
Modelica.SIUnits	Type definitions with SI standard names and units.
Modelica.Electrical	Common electrical component models.
Modelica.Blocks	Input/output blocks for use in block diagrams.
Modelica.Mechanics.Translational	1D mechanical translational components.
Modelica.Mechanics.Rotational	1D mechanical rotational components.
Modelica.Mechanics.MultiBody	MBS library—3D mechanical multibody models.
Modelica.Thermal	Thermal phenomena, heat flow, etc. components.
...	...

2.20 Implementation and Execution of Modelica

1. Modelica source code is parsed and converted into an internal representation
2. Representation is analyzed:
 - Type checking
 - Classes are inherited and expanded
 - Modifications and instantiations are performed
 - Connect equations are converted to standard equations
 - **Result is a flat set of equations, constants, variables, and function definitions**
 - No trace of object-oriented structure remains
3. All of the equations are topologically sorted according to the data-flow dependencies between the equations
 - In case of **DAEs: sorting, manipulation of equations to convert the coefficient matrix into block lower triangular form**
 - **Optimizer module: algebraic simplification algorithms, symbolic index reduction methods**
 - Elimination of most of the equations
 - **Keeping minimal set of equations that will be solved numerically**
4. **Independent equations in explicit form are converted to assignment statements**
 - Possible since the equations have been sorted and an execution order has been established for evaluation of the equations in conjunction with the iteration steps of the numeric solver
 - **Strongly connected set of equation: set is transformed by a symbolic solver** -> performs a number of algebraic transformations to simplify the dependencies between the variables (can solve a differential equation if it has a symbolic solution)
5. **C code is generated and linked with a numeric equation solver** that solves the remaining, drastically reduced, equation system
6. **Approximations to initial values are taken from the model definition** or are interactively specified by the user
 - Numeric solver for DAEs computes the values of the variables during the specified simulation interval
 - Result of the dynamic system simulation is a set of functions of time
 - Functions can be displayed as graphs and/or saved

Open Modelica Compiler

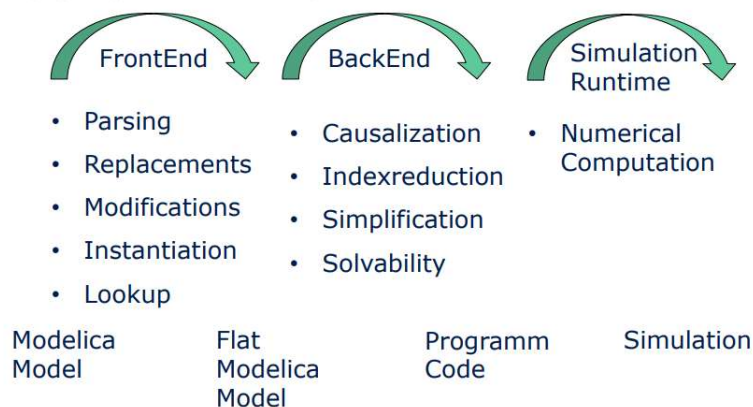
Freitag, 25. Juli 2025 10:55

Datum	24.07.2025
Kategorie	Tool Analysis
Quelle	ecp19157071.pdf , OpenModelica Compiler — OpenModelica User's Guide v1.26.0-dev-207-g710d7e10735 documentation , Titel
Arbeitspaket	Specification - Analysis Existing Tools

Open Modelica Compiler (OMC)

- Consists of frontend, backend, code generation and the runtimes

(Open)Modelica Compiler:



1. Lexical Analysis:

- Keywords, operators, and identifiers are extracted from the model

2. Parsing:

- Abstract syntax tree represented in Meta-Modelica is created from the operators and identifiers

3. Semantic Analysis

- abstract syntax tree gets tested for semantic errors

4. Intermediate Representation

- translation of the abstract tree to an intermediate representation called SCode in MetaModelica
- This is further processed by the frontend producing DAE intermediate representation code

5. Symbolic Optimization backend

- Intermediate representation gets optimized and preprocessed

6. Code Generation

- Executable code gets generated from the low level intermediate representation

Frontend Modules

Motivation

- Traditional generation of simulation code
 - system model is flattened (expanded), i.e., reduced to a large system of scalar equations
 - then performing structural analysis and code generation
 - inefficient when there are many similar components, that only differ by their parameters because structural analysis and generated code will be highly redundant
- Arrays of models, or even multiple instances of the same model, which only differ by the values of their parameters, should not be flattened to their scalar equations
 - rather handled in an efficient way through the code generation process
 - structural analysis and symbolic simplification of models which are instantiated multiple times should be performed only once instead of many times
 - If there are 1000 instances of the same component in a model, code should be generated for the equations of one of them only and then called 1000 times
 - requires fundamental changes to the current flattening to scalar equations before starting code generation

New Frontend Architecture

- Implemented in MetaModelica 3.0
 - combines Modelica Features with functional language features
 - consists of packages and uniontypes defining encapsulated data structures and functions that operate on the defined data
- Does not require verbose listing of all components or named component access of the record in the pattern matching
- Arrays of basic types and arrays of models are not expanded until the scalarization phase
- Old frontend: takes one component and flattens it to a list of variables and equations for that single component before continuing with the next component
 - involves instantiation, scalarization of arrays, typing, etc.
 - components in models often have dependencies: components sometimes need to be flattened partially or fully out of order (parts of the model are flattened multiple times)
- New approach: each frontend phase processes the whole model before the model is passed to the next phase

1. Instantiation

- takes all libraries and models in form of SCode and builds an instance tree for that model
- Instance tree consists of
 - class instance corresponding to the model as the root node
 - component instances of the class as child nodes themselves have component instances
 - instance tree is used for name look up
- Tasks of the instantiation
 - partial instantiation: constructing a lookup-tree
 - expansion: all inherited names and names of local components are added to the lookup tree (declaration order needs to be preserved)
 - full instantiation: components of the classes are instantiated (looking up the type of each component and instantiating it)
 - modifiers are also associated with the elements they modify, and redeclares are applied
 - Allows that the lookup tree for a class is only constructed once and then reused for all instances of that particular class

2. Expression Instantiation

- Expression in the compiler: numbers, strings, unary/binary operator expressions, named references to elements
 - used to represent equations and algorithms, modifiers and array dimensions
- Old Frontend:
 - represents names used in expressions as nested structure
 - each node contains the referenced element's name and type, subscripts used, and for qualified names a reference to the next part of the name
- New Frontend:
 - instead of storing the element's name it stores a reference to the element's instance tree node (which itself contains the name)
- Consequence:
 - frontend always has direct access to the referenced elements
 - frontend needs to find the correct instance node during name lookup (can be tricky during instantiation since names can refer to elements that have not yet been instantiated)
- Expression instantiation phase instantiates all expressions in that instance tree
 - involves looking up the names used in expressions and associating them with the correct nodes in the instance tree
 - in case of function calls: this triggers instantiation of the called functions (mostly involves instantiating the function as a normal class)

3. Typing

- traverses the instance tree and determines the type of all components and expressions
- Done in 2 stages
 - typing of components
 - typing of expressions

- Typing of components
 - determining the type of each component in the instance tree
 - for components of basic types (integer, real) this is trivial
 - for composite types (instances of models, blocks, records) it means typing each child of the instance tree node and constructing a type from them
 - typing of dimensions for array component and classes
- Typing of expressions
 - typing binding equations, equations, and algorithms
 - checking that expressions are type compatible (with the components they belong to)
 - expressions can be typed with the assumption that all components are already typed and acyclic
 - treats array component as one component
 - checks `real[3]==real[3]` instead of 3 times `real=real`
 - superior efficiency with more array elements

4. Flattening

- traverses the instance tree and flattens the tree into a flat model that consists of
 - a list of variables
 - a list of equations
 - a list of algorithms
- involves prefixing component names and element name references in expressions with the names of their parents in the instance tree
 - to make sure that all variables in the flat model have unique names
- collects all the connect equations in the model and inserts the required equations generated from the connections into the flat model
- Unrolling of for-equations into scalar equations (can be disabled)

5. Constant Evaluation

- traverses the flat model and replaces references to constants with the values bound to those chosen constants
- constant evaluation phase makes sure that all occurrences in the model are replaced with the parameter's value (structural parameters like array dimensions)

6. Simplification

- traverses the flat model and simplifies expressions, equations and algorithms
- trivial simplifications such as evaluating unary and binary operations involving numerical literals
 - $1+1=2$
- structural changes such as removing for-loops with zero-sized iteration ranges

7. Scalarization

- expand array variables and equations into separate scalar variables and equations
- would be not necessary for the new frontend, but the old backend expects it

8. Function collection

- flat model contains only the variables, equations and algorithms of the model
- functions used in the model are stored in the instance tree nodes corresponding to the function classes
- backend expects to get a binary search tree containing the functions that are used in the model
- goes through the model and collects all functions that are used in the model into a function tree
 - explicitly called functions
 - record structures for all record instances

9. DAE conversion

- converts the flat model and the function tree to the DAE structure expected by the backend
- serves as interface between new frontend and backend
- straight-forward since the DAE structure is mostly a subset of the data structures used by the new frontend

Backend Modules

1. Pre-Optimization:

- Partitioning

- Alias removal

2. Causalization

- Matching
- Sorting
- Index Reduction

3. Post-Optimization

- Tearing
- Jacobian

Backend DAE Info

- Possible to get additional information from the Backend Modules
 - Number of equation and variables
 - Number of States
 - Information about initialization and simulation system
 - equation types
 - equation system details

Code generation

- from the low-level intermediate representation from the backend code will be generated
- default code generation target is C and offers the largest model coverage
- Alternative: C++ which can produce significant faster executables in some cases
- Target language can be changes with compiler flag `--SimCodeTarget`
- depending on the target, the compiler will write code and compile everything into a single simulation executable

Simulation Runtimes

- generated code is linked with the appropriate runtime

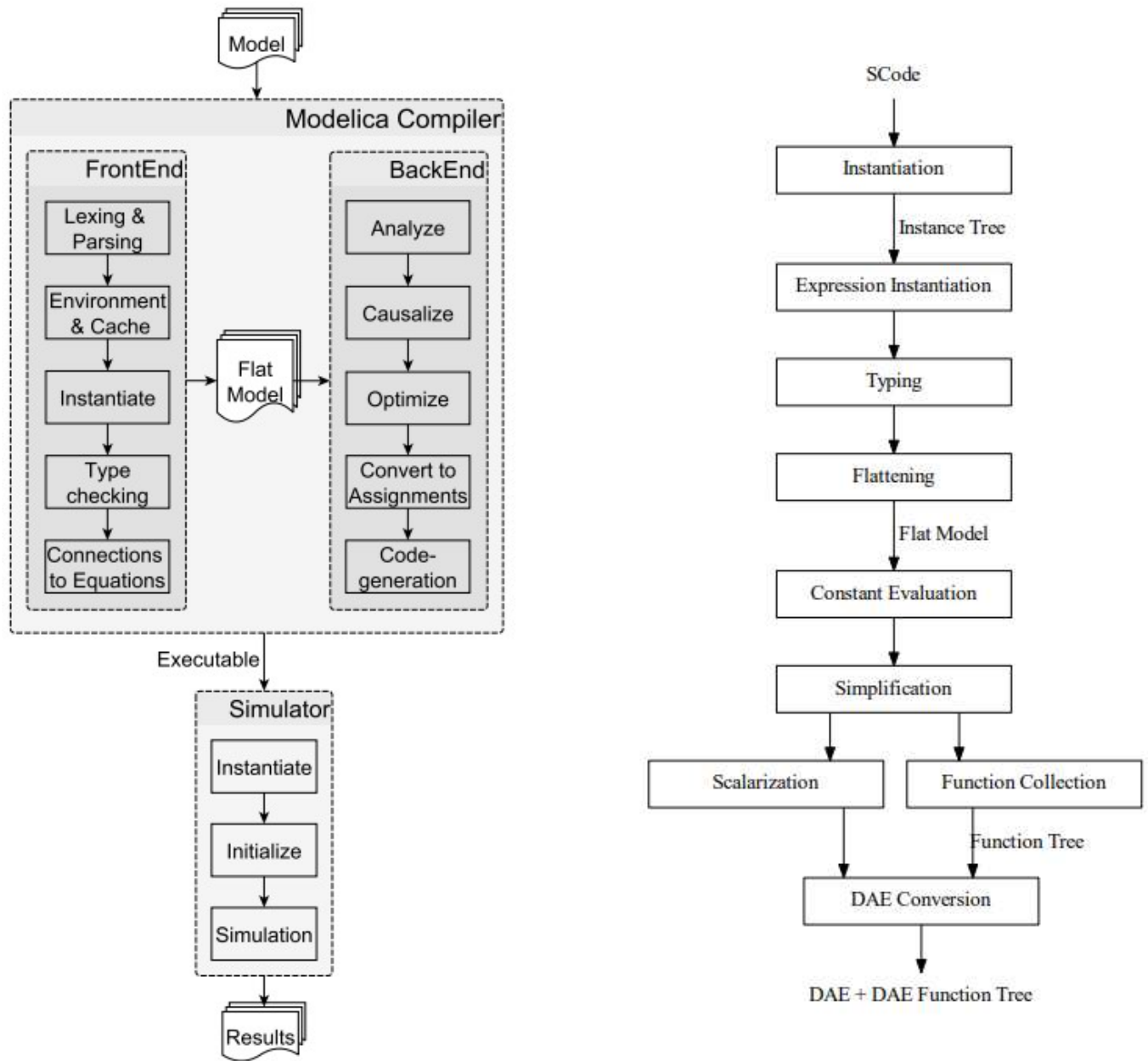


Figure 1. Frontend phases.

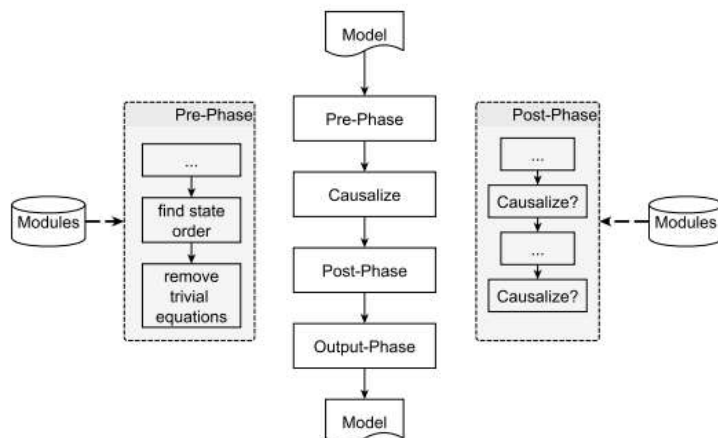


Abbildung 6.2: BackEnd Pipeline

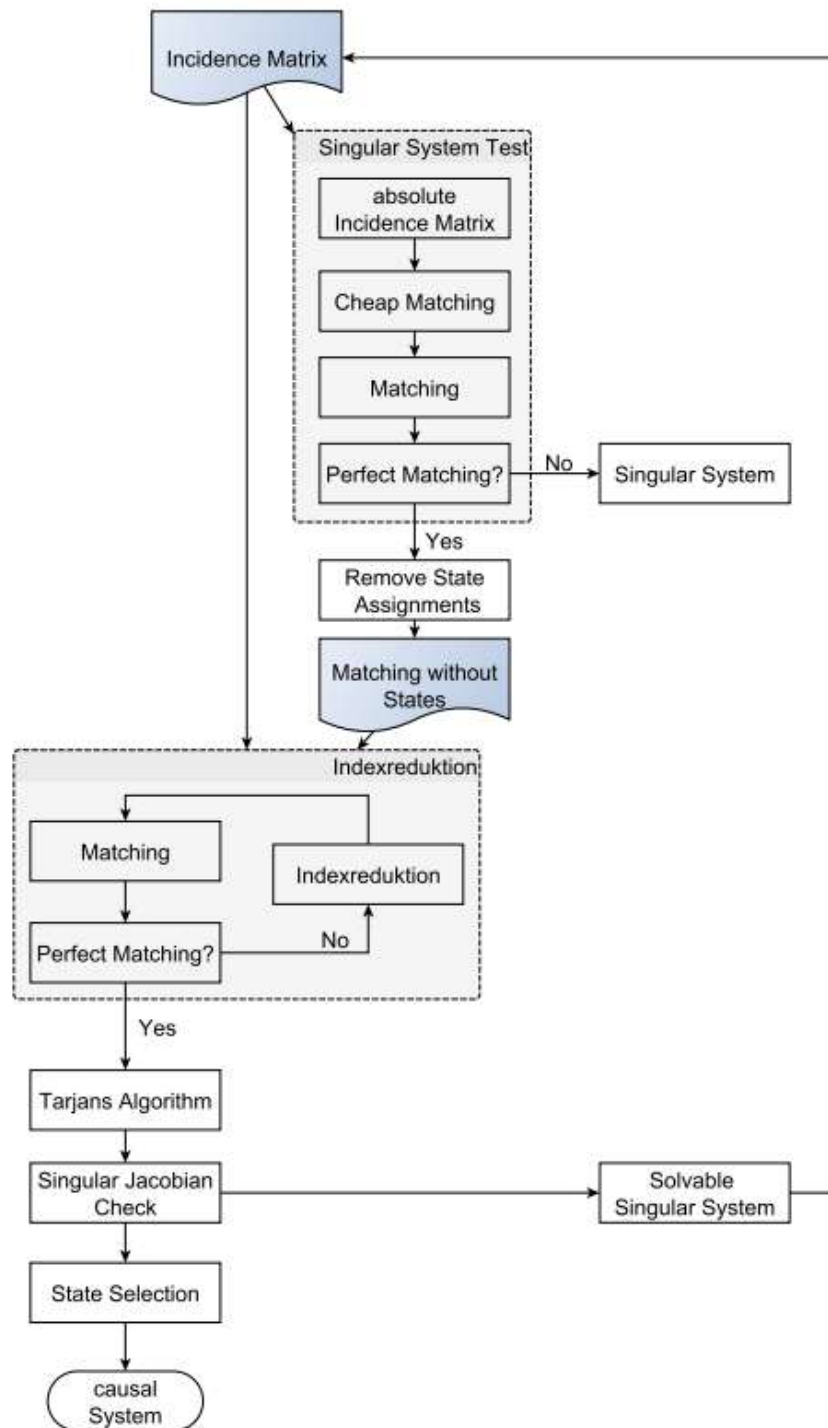


Abbildung 6.3: Ablaufschema zur Kausalisierung